

Practical 7: Implementation of making a change problem using dynamic programming**Code:**

```
#include <iostream>

#include <vector>

#include <climits>

using namespace std;

int minCoins(vector<int>& coins, int amount) {

    vector<int> dp(amount + 1, INT_MAX);

    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {

        for (int coin : coins) {

            if (coin <= i && dp[i - coin] != INT_MAX) {

                dp[i] = min(dp[i], dp[i - coin] + 1);

            }

        }

    }

    return dp[amount] == INT_MAX ? -1 : dp[amount];

}

int main() {

    int n, amount;

    cout << "Enter number of coins: ";

    cin >> n;
```

```
vector<int> coins(n);

cout << "Enter coin denominations: ";

for (int i = 0; i < n; i++)

    cin >> coins[i];

cout << "Enter amount: ";

cin >> amount;

int result = minCoins(coins, amount);

if (result == -1)

    cout << "Change cannot be made";

else

    cout << "Minimum number of coins = " << result;

return 0;

}
```

Output:

```
Enter number of coins: 3
Enter coin denominations: 5 6 7
Enter amount: 12
Minimum number of coins = 2
```

Signature:_____

Date:_____

Practical 8: Implementation of Graph and Searching (DFS and BFS).**Code:**

```
#include <iostream>

#include <vector>

#include <queue>

using namespace std;

class Graph {

    int V;

    vector<vector<int>>> adj;

public:

    Graph(int v) {

        V = v;

        adj.resize(V);

    }

    void addEdge(int u, int v) {

        adj[u].push_back(v);

        adj[v].push_back(u); // Remove for directed graph

    }

    void DFSUtil(int node, vector<bool>& visited) {

        visited[node] = true;

        cout << node << " ";

        for (int neighbour : adj[node]) {

            if (!visited[neighbour])
```

```
        DFSUtil(neighbour, visited);
    }
}
```

```
void DFS(int start) {
    vector<bool> visited(V, false);
    DFSUtil(start, visited);
}
```

```
void BFS(int start) {
    vector<bool> visited(V, false);
    queue<int> q;

    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int node = q.front();
        q.pop();

        cout << node << " ";

        for (int neighbour : adj[node]) {
            if (!visited[neighbour]) {
                visited[neighbour] = true;
                q.push(neighbour);
            }
        }
    }
}
```

```
    }  
}  
};
```

```
int main() {  
    Graph g(6);  
  
    g.addEdge(0, 1);  
    g.addEdge(0, 2);  
    g.addEdge(1, 3);  
    g.addEdge(1, 4);  
    g.addEdge(2, 5);  
  
    cout << "DFS Traversal: ";  
    g.DFS(0);  
  
    cout << "\nBFS Traversal: ";  
    g.BFS(0);  
  
    return 0;  
}
```

Output:

```
DFS Traversal: 0 1 3 4 2 5  
BFS Traversal: 0 1 2 3 4 5
```

Signature:_____

Date:_____

Practical 9: Implement the LCS problem.**Code:**

```
#include <iostream>

#include <vector>

#include <algorithm>

using namespace std;

int main() {

    string X, Y;

    cout << "Enter first string: ";

    cin >> X;

    cout << "Enter second string: ";

    cin >> Y;

    int m = X.length();

    int n = Y.length();

    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

    for (int i = 1; i <= m; i++) {

        for (int j = 1; j <= n; j++) {

            if (X[i - 1] == Y[j - 1])

                dp[i][j] = dp[i - 1][j - 1] + 1;

            else

                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);

        }

    }
```

```
}

cout << "Length of LCS = " << dp[m][n] << endl;

string lcs = "";

int i = m, j = n;

while (i > 0 && j > 0) {
    if (X[i - 1] == Y[j - 1]) {
        lcs += X[i - 1];
        i--;
        j--;
    }
    else if (dp[i - 1][j] > dp[i][j - 1])
        i--;
    else
        j--;
}

reverse(lcs.begin(), lcs.end());

cout << "LCS = " << lcs << endl;

return 0;
}
```

Output:

```
Enter first string: stuvbw  
Enter second string: tuwjhb  
Length of LCS = 3  
LCS = tuw
```

Signature:_____

Date:_____

Practical 6: Implementation of a knapsack problem using dynamic programming.**Code:**

```
#include <iostream>

#include <vector>

#include <algorithm>

using namespace std;

int knapsack(int capacity, const vector<int>& weight,

            const vector<int>& value, int n) {

    // dp[i][w] = maximum value using first i items

    // with knapsack capacity w

    vector<vector<int>>> dp(n + 1,

                          vector<int>(capacity + 1, 0));

    for (int i = 1; i <= n; i++) {

        for (int w = 0; w <= capacity; w++) {

            // Item can be included

            if (weight[i - 1] <= w) {

                dp[i][w] = max(

                    dp[i - 1][w],

                    value[i - 1] +

                    dp[i - 1][w - weight[i - 1]]

                );

            }

            // Item cannot be included

            else {
```

```
        dp[i][w] = dp[i - 1][w];
    }
}
}

return dp[n][capacity];
}

int main() {
    int n, capacity;

    cout << "Enter number of items: ";
    cin >> n;

    vector<int> weight(n);
    vector<int> value(n);

    cout << "Enter weights:\n";
    for (int i = 0; i < n; i++)
        cin >> weight[i];

    cout << "Enter values:\n";
    for (int i = 0; i < n; i++)
        cin >> value[i];

    cout << "Enter knapsack capacity: ";
    cin >> capacity;
```

```
cout << "Maximum value = "  
  
    << knapsack(capacity, weight, value, n)  
  
    << endl;  
  
return 0;  
  
}
```

Output:

```
Weights: 10 20 30  
Values:  60 100 120  
Capacity: 50
```

```
Maximum value = 220
```

Signature:_____

Date:_____

Practical 10: Implement LCS problem.**Code:**

```
#include <iostream>

#include <vector>

#include <algorithm>

using namespace std;

int LCS(string X, string Y) {

    int m = X.length();

    int n = Y.length();

    // dp[i][j] = length of LCS of X[0...i-1] and Y[0...j-1]

    vector<vector<int>>> dp(m + 1, vector<int>(n + 1, 0));

    for (int i = 1; i <= m; i++) {

        for (int j = 1; j <= n; j++) {

            if (X[i - 1] == Y[j - 1]) {

                dp[i][j] = 1 + dp[i - 1][j - 1];

            }

            else {

                dp[i][j] = max(dp[i - 1][j],

                               dp[i][j - 1]);

            }

        }

    }

}
```

```
        return dp[m][n];
    }

int main() {
    string X, Y;

    cout << "Enter first string: ";
    cin >> X;

    cout << "Enter second string: ";
    cin >> Y;

    cout << "Length of LCS = " << LCS(X, Y) << endl;

    return 0;
}
```

Output:

```
Enter first string: ABCDGH  
Enter second string: AEDFHR
```

```
Length of LCS = 3
```

Signature:_____

Date:_____